

altairsim — Recipe: a CP/M Altair, built from nothing

2026-09-23 · b1a286b

You will start with an empty chassis, fit six boards by hand, boot CP/M, save the machine to a file, quit, and load it back. It takes about five minutes and you type every line yourself.

Every recipe in this folder has the same four beats:

altairsim -n — an empty backplane. **Fit the boards** — `BOARDS ADD`, `REGION ADD`, `SET`, `MOUNT`, `STARTUP ADD`. **CONFIG SAVE** — write the machine you just built to a file. **Quit**, **altairsim -n** again, **CONFIG LOAD** — and the machine comes back.

Work in a copy of `examples/cpm/`, because that folder has the disk this recipe boots. A path you type is relative to the machine's folder, so being in the right one is the whole trick.

```
$ cd examples/cpm
```

1. An empty chassis

```
$ altairsim -n
AltairSim X.Y.Z -- no CPU in the backplane; the monitor is the bus master.
machine: none.  HELP for commands.
altairsim> BOARDS
(empty backplane)
```

`-n` is nothing at all: no memory, no processor, no serial port. Everything from here is a board you chose to fit, which is why it is the honest place to start.

2. Fit the boards

Six of them. Each line is a board going into the backplane, and the machine answers every one.

```
altairsim> BOARDS ADD fp fp0
fp0: fp added
altairsim> BOARDS ADD 8080 cpu0
cpu0: 8080 added
altairsim> BOARDS ADD 2sio sio0 port=10
sio0: 2sio added
altairsim> SET sio0:a connect=console
sio0:a: connect=console
altairsim> BOARDS ADD dcdd dsk0 port=08 drives=4
dsk0: dcdd added
altairsim> BOARDS ADD memory mem0 fill=random
mem0: memory added
```

What each one is, in order: the front panel, the 8080 processor, the 88-2SIO serial board at port `10` with its first port wired to your terminal, the 88-DCDD floppy controller at port `08` with four drives, and a memory board.

Three things about that list are worth knowing before you go further.

A board's settings ride along on the `BOARDS ADD` line, as `port=10` does above. You can set them afterwards with `SET` instead — it is the same setting either way.

`SET` takes one property per command. This looks reasonable and is not:

```
altairsim> SET sio0:a connect=console baud=9600
SET: unexpected 'baud=9600'
```

Two lines. One property each.

A board has to exist before you can `SET` on it. Reverse those two lines and you get

```
no board 'sio0'. BOARDS shows what is in the machine.
```

The memory board is empty until you say what is in it

Fitting a memory board does not give you any memory. The board is the card; what is *on* the card is a list of regions, and you add them one at a time:

```
altairsim> REGION ADD mem0 type=ram at=0 size=56K
mem0:0: ram 0000-DFFF 56K
altairsim> REGION ADD mem0 type=rom at=FF00 mount=builtin:dbl
mem0:1: rom FF00-FFFF builtin:dbl
```

56K of RAM from 0000, and the Altair disk boot loader in ROM at FF00. builtin:dbl is compiled into altairsim — there is no file to find.

Leave that second line out and the machine has nothing to boot from. It is the most common way to end up with a machine that looks complete and does nothing.

3. The terminal, the disk, and the keystroke that starts it

```
altairsim> SET CONSOLE bsdel=bs
console: bsdel=bs
altairsim> MOUNT dsk0:drive0 cpm22b23-56k.dsk
dsk0:drive0: mounted cpm22b23-56k.dsk
altairsim> STARTUP ADD RUN FF00
added: RUN FF00
altairsim> SET MACHINE name=mycpm
machine: name=mycpm
```

bsdel=bs makes your Backspace key erase at the CP/M prompt: a modern terminal sends DEL and CP/M's line editor wants BS. It is a property of your terminal, not of the machine.

MOUNT puts the disk in drive 0 — the name with no directory in it, because the file is lying beside the machine in this folder.

STARTUP ADD records a command to run when this machine loads. There is no BOOT command in altairsim, because there was no BOOT switch on the Altair: to start the disk loader the operator set FF00 in the address switches, pressed EXAMINE to load it into the program counter, and then RUN. RUN is those last two in one — it examines the address and starts the processor there. A bare RUN with no address does not touch the program counter; it starts the processor wherever it already is, which is how you resume after a STOP.

SET MACHINE name= names the machine. A machine built from -n is called none until you say otherwise, and the name is what SHOW MACHINE prints, what CONFIG SAVE writes into the file, and what another

machine file's `base =` would refer to.

4. Look at what you built

SHOW MACHINE is the machine you have built — its name, the startup list, and every board in the backplane:

```
altairsim> SHOW MACHINE
name      mycpm
startup
          RUN FF00

  ID  TYPE  I/O  UNITS  MEMORY
  ---  -
fp0   fp    FF   -
cpu0  8080   -   1 cpu: 8080
sio0  2sio  10,12  2 serial: a*, b
dsk0  dcdd  08,09,0A  4 disk: drive0, drive1(empty), drive2(empty), drive3(empty)
mem0  memory -   1 rom: rom0
                                0000-DFFF ram 56K
                                FF00-FFFF rom dbl

phantom:all

* holds the console
```

That is a 56K CP/M Altair. **BOARDS** prints the same table without the name and the startup list, and **SHOW <id>** — **SHOW mem0**, **SHOW cpu0** — prints everything one board has, with its current value and what it will accept.

5. Save it

```
altairsim> CONFIG SAVE mycpm.toml
saved mycpm.toml
altairsim> QUIT
```

CONFIG SAVE writes the machine that is actually in the backplane right now — every board, every setting, including the ones you did not touch. **This is the fastest way to write a machine file**: build it at the prompt until it works, then save it.

The file opens with the name you gave it, and the startup line you recorded:

```
[machine]
name      = "mycpm"
startup  = [
    "RUN FF00",
]
```

You will notice the rest of the file is longer than you expected — it carries settings you never typed (`idle` , `dcd` , `cts` , `phantom` , `seed` , and the rest). That is deliberate: it writes down the whole machine, so loading it gives you exactly this machine back and not a newer program's idea of a default.

6. Load it into an empty chassis

Start again with nothing, and hand it the file:

```
$ altairsim -n
altairsim> CONFIG LOAD mycpm.toml
loaded mycpm.toml: 5 board(s)
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

That is CP/M, off the disk, on the machine you built. Type `DIR` to see what is on it, and Ctrl-E to get back to the `altairsim>` prompt.

`CONFIG LOAD` replaces the whole machine — it is the same thing as naming the file on the command line:

```
$ altairsim mycpm.toml
```

It is all or nothing. The new machine is built off to one side first, so a file with a mistake in it leaves you exactly where you were rather than halfway between two machines.

What to do next

Compare your file with `examples/cpm/cpm22-buffered.toml` . They boot the same machine, and the shipped one is a few lines long, because it says `base = "default"` and then writes down only what makes it different. That is the next thing to learn, and `recipes/from-a-builtin.pdf` is the walkthrough for it.

And look at what is already on the shelf before you build another one by hand. `SHOW MACHINES` lists every machine built into altairsim, and `SHOW MACHINE <name>` opens one up without loading it. `SHOW MACHINE default` is the machine you just built, board for board, with one addition — the host bridge, which moves files between the guest and your own disk. Building it yourself was worth doing once; now you know what that one line is made of. The other recipe starts there.